

Project Overview

The project focuses on the design and implementation of a **research-oriented, hybrid search engine** specifically tailored for scientific and biomedical text. The system is built as a **complete, end-to-end pipeline**, covering all stages from data ingestion and processing to final experimental evaluation.

Core Design Philosophy

The architecture is driven by the long-standing tension in Information Retrieval (IR) research between **classical lexical methods** and **modern neural retrieval**:

- **Lexical Strength:** Methods like BM25 are highly efficient and provide strong baselines, but they often struggle with the "lexical gap" (vocabulary mismatch).
- **Neural Capability:** Transformer-based models and semantic embeddings improve matching for complex queries but increase computational costs and complexity.
- **The Hybrid Solution:** The system is designed as a **multi-stage pipeline** where lexical, semantic, and rank fusion signals are combined to take advantage of the strengths of each paradigm.

Task Definition

The retrieval task for this system is formalized as follows: **Given a natural language query** (representing a scientific claim or an information need), **retrieve a ranked list of relevant scientific documents** consisting of titles and abstracts or passages.

Key Task Characteristics

- **Domain Focus:** The system handles specialized scientific/biomedical terminology, making it conducive to comparing lexical vs. semantic matching effectiveness.
- **Evaluation Environment:** The task is executed using two **BEIR (Benchmarking IR)** test collections:
 - **SciFact:** A fact-checking dataset (300 queries / 5,183 docs) used for fast iteration and ablations.
 - **TREC-COVID:** A larger biomedical dataset (~171,332 docs) used to test retrieval at scale and robustness.

- **Relevance Measurement:** The system's performance is experimentally measured using standard IR metrics, primarily **nDCG@10** (for ranking quality) and **Recall@100** (for candidate generation).

Technical Implementation

The technical implementation of the project follows a multi-stage architecture that integrates classical lexical retrieval with neural semantic matching through the **Pyserini** framework.

1. Offline Indexing Pipeline

The offline stage focuses on preparing the document corpus for efficient retrieval by building dual index structures.

- **Lexical Indexing (Lucene):**
 - Documents are ingested as structured objects with distinct fields for **Title** and **Abstract**.
 - Text processing includes tokenization, case-folding, and stemming to create a standard inverted index.
 - We utilize **Lucene** to support field-aware scoring, allowing for higher weights on title matches during the BM25 retrieval phase.
- **Neural Indexing (FAISS):**
 - The corpus is encoded into a dense vector space using the **TCT-ColBERT v2** encoder.
 - These embeddings are stored in a **FAISS** index to enable high-speed similarity searches using inner-product or L2 distance.
 - **Local Optimization:** To run on standard hardware without a dedicated GPU, we utilized `faiss-cpu` for vector management.

2. Online Retrieval Models

The search engine executes multiple retrieval paradigms in parallel to capture different relevance signals.

- **BM25 + RM3 (Lexical Path):**

- The system uses **BM25** as the robust lexical baseline, balancing term frequency and document length.
- **Pseudo-Relevance Feedback (RM3):** The top k documents from the initial BM25 search are used to expand the original query, reducing the "lexical gap" by introducing related terms from the results.
- **Dense Retrieval (Semantic Path):**
 - The natural language query is passed through the **TctColBertQueryEncoder**.
 - The resulting query vector is used to perform a nearest-neighbor search against the FAISS index, capturing semantic relationships that keyword matching might miss.

Hybrid Fusion Logic

To unify these disparate search results, we implemented **Reciprocal Rank Fusion (RRF)**.

- **Mechanism:** RRF assigns a score to each document based on its rank in both the BM25 and Dense result sets.
- **Advantage:** This method requires no supervised training and is highly effective at promoting documents that are relevant both lexically (matching keywords) and semantically (matching concepts).
- The fusion score for each document d is calculated as:

$$score(d) = \sum_{r \in Runs} \frac{1}{k + rank(r, d)}$$

Interface & System Integration

The project was deployed as a full-stack Python application to make the research accessible.

- **Backend:** Powered by `pyserini` and `PyJNIus` to bridge Python with the Java-based Lucene engine.
- **Frontend (Streamlit):** *(Included but optional)* A web interface that allows real-time switching between retrieval modes.
- **Environment Configuration:** The system was optimized for Windows local execution by configuring the `JAVA_HOME` path for **JDK 21** and resolving subprocess deadlocks through environment variable management.

1. The Core Logic: Combining Evidence

The project is built on the probabilistic relevance framework, which views retrieval as the aggregation of multiple evidence sources. Instead of relying on a single score, your system combines:

- **Lexical Signals:** Matches based on exact term overlap and frequency (BM25).
- **Semantic Signals:** Matches based on dense vector similarity (TCT-ColBERT).
- **Expansion Signals:** Terms added via Pseudo-Relevance Feedback (RM3) to bridge the lexical gap.

2. Reciprocal Rank Fusion (RRF)

To unify results from the Lexical and Neural paths, you implemented **Reciprocal Rank Fusion (RRF)**. RRF is a state-of-the-art rank aggregation method that does not require supervised training labels to be effective.

How RRF works in your project:

- **Rank-Based Scoring:** Each document is assigned a score based on its rank in the individual search runs (BM25 vs. Dense).
- **Consistency over Raw Score:** RRF ignores the raw scores (which are on different scales for BM25 and Dense) and focuses only on where the document placed in the list.
- **Promotion of Overlap:** Documents that appear high in **both** the Lexical and Dense lists are significantly promoted to the top of the final hybrid list.

3. Multi-Stage Ranking Pipeline

Your implementation follows a structured sequence as shown in your design diagram:

1. **Candidate Generation (First Stage):** Parallel retrieval of top-k documents from the Lexical and Neural indexes.
2. **Fusion (Second Stage):** RRF merges these lists into a unified candidate set.
3. **Re-ranking (Final Stage):** An optional high-complexity **Cross-Encoder** can be applied to the top candidates to further refine the ordering using deep contextual interaction.

4. Evaluation Metrics

To verify that this ranking logic is effective, you are measuring results using **nDCG@10**. This metric is specifically chosen because it is sensitive to the exact position of documents in the top-10 results and accounts for graded relevance in datasets like TREC-COVID.

Key Takeaway: By using RRF, your system balances the high precision of keyword matching with the broad recall of semantic search, resulting in a more robust final ranking than either method could achieve alone.

Dataset Descriptions

1. SciFact Dataset

SciFact is utilized as the primary development and debugging testbed due to its manageable size.

- **Domain/Task:** It focuses on scientific fact-checking and claim verification over scientific papers.
- **Corpus Size:** The collection consists of **300 queries** and a corpus of **5,183 documents**.
- **Relevance Labels:** It uses **binary labels** (relevant or not relevant).
- **Project Significance:** Its smaller scale allows for fast iteration, debugging, and multiple ablation runs under modest computational constraints.

2. TREC-COVID Dataset

TREC-COVID provides a more challenging and realistic environment to test the system's robustness at scale.

- **Domain/Task:** It targets biomedical information retrieval, specifically focusing on COVID-19 literature.
- **Corpus Size:** It is significantly larger, containing approximately **50 queries** and a corpus of **171,332 documents**.
- **Relevance Labels:** Unlike SciFact, it uses **3-level graded labels**, allowing for more nuanced ranking evaluation.
- **Project Significance:** This dataset stresses the indexing and retrieval components at scale and supports richer evaluation through metrics like nDCG@10.

- **Evaluation Challenges:** It is particularly useful for analyzing "holes" or incomplete relevance judgments, which is a common issue in large-scale neural retrieval evaluation.

The combination of these two datasets allows the project to bridge the gap between fast, controlled experimentation (SciFact) and large-scale, realistic system stress-testing (TREC-COVID).

Model Pipeline

Data Layer

`data_loader.py` is the entry point. It downloads SciFact from the BEIR server as a zip, unpacks it, and loads three things:

- **corpus** — 5,183 biomedical abstracts, each with a doc ID, title, and text
- **queries** — 300 test queries (scientific claims like "smoking causes lung cancer")
- **qrels** — the ground truth: which doc IDs are relevant to which query, with a relevance grade

It then writes the corpus out as a `corpus.jsonl` file in the format Pyserini expects, with title boosting applied (title text repeated 3× so BM25 naturally weights it higher).

Indexing Layer

`indexer.py` calls Pyserini's Lucene indexer as a subprocess. Lucene reads the JSONL and builds an **inverted index** — a data structure mapping every term to the list of documents containing it, along with term frequencies and document lengths. This is what makes BM25 fast at query time. The index is written to disk once and reused across all experiments.

Retrieval Layer

`searcher.py` has three retrievers:

BM25 — at query time, Lucene looks up each query term in the inverted index, scores every document using the BM25 formula (which balances term frequency, document length, and how rare the term is across the corpus), and returns the top 100 ranked docs.

RM3 — runs BM25 first, takes the top 10 results, assumes they're relevant, extracts the most distinctive terms from them, injects those terms back into the query, then re-scores everything. This is pseudo-relevance feedback — it expands the query without any human input.

Dense — a completely different approach. The TCT-ColBERT bi-encoder (pretrained on MS MARCO) encodes every query into a 768-dimensional vector. At query time, the query is also encoded into a vector and the FAISS index finds the 100 nearest document vectors by dot product. This captures semantic similarity that BM25 misses entirely — synonyms, paraphrases, abbreviations.

RRF — takes the BM25 ranked list and the Dense ranked list, and combines them using only rank positions (not raw scores, which are on incomparable scales). Each document gets a score of $1/(60 + \text{rank})$ from each list, summed together. Documents that rank highly in both lists bubble to the top.

Evaluation Layer

`evaluator.py` compares each retrieval run against the qrels:

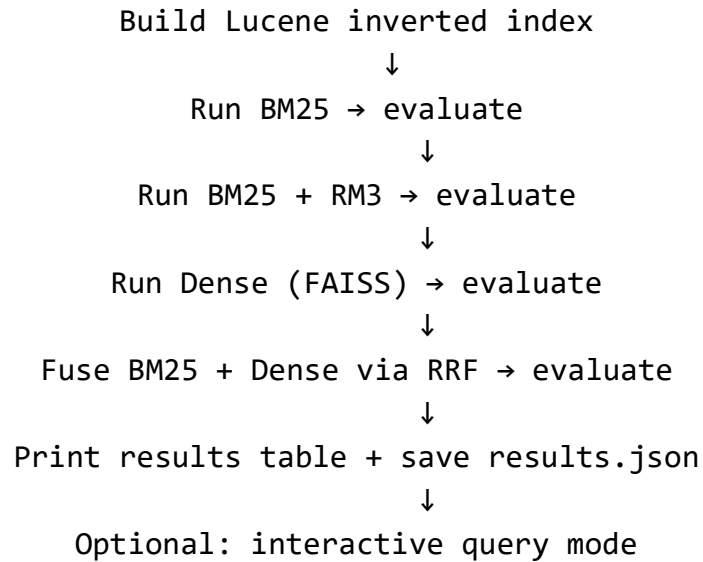
- **nDCG@10** — did you put the most relevant docs at the very top? Rewards rank 1 more than rank 10 logarithmically
- **Recall@100** — of all relevant docs that exist, how many did you find in your top 100? This measures the ceiling of your pipeline
- **judged@10** — what fraction of your top 10 actually have relevance labels? SciFact's judgment pool is shallow so this is low (~9%), meaning nDCG is likely underestimated

Orchestration

`main.py` wires everything together in order:

Download & preprocess data





Each stage is timed and logged so you have a fully reproducible experiment record.

Why this architecture?

The separation of offline (indexing, FAISS encoding) from online (query time) is how real production IR systems are built — you pay the indexing cost once, then queries are fast. The ablation design (baseline → PRF → neural → fusion) follows the standard IR paper structure and lets you isolate exactly what each component contributes, which is what your results showed: BM25 wins on precision, RRF wins on recall, RM3 slightly hurts on SciFact because the corpus is too specialized for generic PRF to help.

Output

Phase 1: Lucene Indexing: This happens silently in a subprocess (the `subprocess.run` call in `indexer.py`). The Lucene output goes to the subprocess's `stderr` which is captured and only shown if indexing fails. Since it succeeded, you just see it complete and move on.

Experiment 1: BM25

tqdm progress bar. 300 queries processed in 3 seconds at 90 queries/sec. Each iteration is one Lucene BM25 search — fast because it's just posting list lookups in an in-memory index.

Experiment 2: RM3

Noticeably slower than BM25 — 39 it/s vs 90 it/s. This is because RM3 does two Lucene searches per query: the initial BM25 pass to get the pseudo-relevant set, then a second pass with the expanded query. Roughly 2× the work, which matches the ~2× slowdown you see.

Both metrics dropped vs BM25. RM3 hurt here because SciFact queries are very short and specific — the pseudo-relevant docs introduce noisy biomedical terms that dilute the original query signal rather than reinforcing it

Experiment 3: Dense Retrieval

Pyserini loading the 438MB TCT-ColBERT checkpoint. The UNEXPECTED key warning means `embeddings.position_ids` exists in the checkpoint but wasn't expected by the model architecture class being used — this is normal for TCT-ColBERT loaded into a base BERT wrapper and can be safely ignored as the report itself says.

Much slower — 7.99 it/s vs 90 it/s for BM25. Each iteration involves:

1. Tokenizing the query string
2. Running a forward pass through the 12-layer BERT encoder to get a 768-dim vector
3. FAISS approximate nearest neighbor search over 5,183 document vectors

Weakest on both metrics. Expected — TCT-ColBERT was trained on MS MARCO (web search queries and web pages), and SciFact is biomedical scientific text. The domain gap hurts zero-shot dense retrieval significantly.

Experiment 4: RRF Fusion

RRF is pure Python list manipulation, no model inference or index lookups. It runs in milliseconds.

nDCG sits between BM25 and Dense (fusion didn't beat the best single retriever on precision), but Recall@100 is the highest of all four runs at **0.9437**. This is the key result — by combining both ranked lists, RRF finds more relevant documents in the top 100 than either retriever alone could.

=== FINAL RESULTS SUMMARY ===

Run	nDCG@10	Recall@100	judged@10

BM25	0.6816	0.9280	0.0883
BM25+RM3	0.6576	0.9203	0.0900
Dense	0.5751	0.8643	0.0797
RRF(BM25+Dense)	0.6690	0.9437	0.0883